Ursinus
COLLEGE

CS375 Spring 2026

Ralph Rostock

Week 8

Implementation & Professional Coding Best Practices

Team Standup Meetings

Last Week

Professional-Style Design Reviews

From Design to Code

- Your design artifacts are inputs, not instructions
  - A class diagram says what exists — not exactly how to write it
  - A sequence diagram suggests call flow — not the exact function signatures
- Implementation reveals what the design got right — and what needs revision
  - This is normal and expected. Design is a hypothesis.
- When your code diverges from the design, update the design
  - Outdated diagrams are worse than no diagrams — they actively mislead
- The translation layer is where the real thinking happens
  - Trace paths through your design and ask: what function does this become?
  - What class owns this behavior? What does "send notification" actually mean in code?

Why Code Quality Matters

- ~80% of a software system's cost is maintenance, not initial development
- Developers spend more time reading code than writing it
  - One estimate: 42% of dev time is spent understanding existing code
- "Working code" is the minimum bar — it's not the goal
  - The goal is code that works and that your team can maintain confidently
- Technical debt is a loan — it accrues interest
  - Every future touch of messy code costs a little more than it should
  - Some debt is intentional (time pressure, prototype). Make it explicit.
- Readable code is a professional courtesy to your teammates
  - Including future-you, two months from now

Technical Debt

Not all debt is the same — and not all of it is a mistake.

	RECKLESS	PRUDENT
DELIBERATE	*"We don't have time for tests"* Worst kind. Conscious corner-cutting.	*"Ship now, refactor after we validate the idea"* Acceptable — if you actually come back.
INADVERTENT	*"We didn't know a better pattern existed"* Common. The cost of learning on the job.	*"Now we know how we should have designed it"* Fine — refactor when you touch it next.

What "paying interest" actually looks like:

- 20 min re-reading code before you can change anything
- A bug fix in one place breaks something unrelated
- Nobody wants to touch the auth module
- Onboarding a new teammate takes days instead of hours
- Every feature takes longer than the last one

Debt isn't just messy code — it's also architecture.

A wrong structural decision made early constrains every decision after it — and no amount of local cleanup fixes it.

What shortcuts have you already made in your project that you know you'll have to come back to?

Coding Standards

- Pick a style guide and follow it consistently as a team
  - Python: PEP 8 | JavaScript: Airbnb or Prettier defaults | Java: Google Style
- Automate enforcement — don't police it manually in code review
  - Linters: ESLint, Pylint, Checkstyle
  - Formatters: Prettier, Black, google-java-format
  - If your linter catches it, a human reviewer shouldn't have to
- Agree on naming conventions before you start coding
  - camelCase vs snake_case, file structure, test file naming
- Write it down — a short CONTRIBUTING.md in your repo
  - Even 5 bullet points eliminates a lot of silent inconsistency

Example Contributing.md

Contributing Guide

Getting Started

1. Clone the repo and install dependencies:

```bash
  git clone https://github.com/your-org/your-project.git
  cd your-project
  npm install
```

2. Copy `.env.example` to `.env` and fill in your local values.

3. Run the app:

```bash
  npm run dev
```

4. Run the tests:

```bash
  npm test```

## Before You Commit

Run these two commands before every commit:

```bash
  npm run lint  # checks for style issues and potential bugs (ESLint)
  npm run format   # auto-formats all files (Prettier)
```

Both must pass cleanly. Your `package.json` should have these scripts configured — see the Setup section below if they're missing.

To set up ESLint and Prettier if not already configured:

```bash
  npm install --save-dev eslint prettier eslint-config-prettier
  npx eslint –init
```
```

Example Contributing.md (continued)

Branching

- `main` is always deployable. Never commit directly to `main`.
- Branch naming: `feature/short-description` or `fix/short-description`
- Keep branches short-lived — aim to merge within a few days.
- Pull from `main` daily to avoid diverging.

Pull Requests

- Every change goes through a PR — no direct pushes to `main`.
- PR title should be descriptive: what changed and why.
- Include in the description:
  - What this PR does
  - How to test it manually
  - Any known limitations or follow-up work
- At least **one approval** required before merging.
- The author merges (after approval) — not the reviewer.

Code Style

- We use **Prettier** for formatting and **ESLint** (Airbnb config) for linting.
- Let the tools decide — don't manually fix formatting that Prettier will rewrite anyway.
- Naming: `camelCase` for variables and functions, `PascalCase` for classes and components, `UPPER_CASE` for constants.
- All exported functions must have a JSDoc comment.
- No commented-out code. Delete it — git remembers it.
- Prefer `const` over `let`. Never use `var`.

Commit Messages

Use the conventional commits format:

```
feat: add user authentication endpoint
fix: handle null response from payment API
refactor: extract order validation into service layer
docs: update README with deployment instructions
test: add unit tests for discount calculation
```

Code Smells

- Long Method — function does too many things; hard to name, hard to test
  - If you can't summarize a function in 5 words, it probably needs splitting
- Magic Numbers — unexplained literal values in logic
  - ❌ if status == 3
  - ✅ if status == OrderStatus.SHIPPED
- Feature Envy — a method that uses another class's data more than its own
  - Often a sign that behavior belongs in the other class
- Shotgun Surgery — one change requires edits in many unrelated places
  - Sign that a concern is scattered; should be centralized
- Dead Code — commented-out blocks, unused variables, unreachable branches
  - Delete it. Git remembers it. It's just noise now.
- Deeply Nested Logic — if inside if inside if inside for loop
  - Early returns and helper functions almost always improve this

Code Smells in Practice – Before & After

Before – several smells present:

```
def proc(u, t):
    if t == 1:
        if u.age >= 18:
            if u.country == "US":
                u.status = 1
                db.save(u)
                return True
    return False
```

After – extracted, renamed, readable:

```
STANDARD_REGISTRATION = 1

def register_user(user, reg_type):
    if reg_type != STANDARD_REGISTRATION:
        return False
    if not is_eligible(user):
        return False
    activate_account(user)
    return True

def is_eligible(user):
    return user.age >= 18 and user.country == "US"

def activate_account(user):
    user.status = "active"
    db.save(user)
```

Smells fixed: magic number, deep nesting, long method, unclear names, mixed concerns

Git & PRs – Refresher and What Matters Right Now

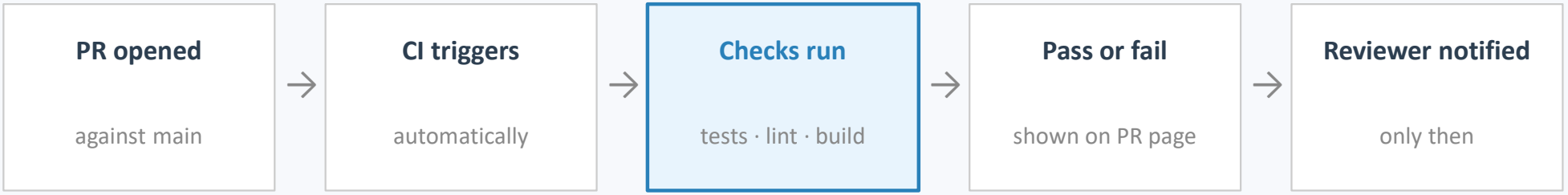
You already know how this works. A few reminders for the implementation phase:

- main is always deployable — branch for every feature or fix
- Commit messages should explain what changed and why
  - ✅ "fix: handle null user in auth guard"
  - ❌ "stuff" / "WIP" / "asdfg"
- Pull from main every day you write code — avoid diverging
- Keep PRs small and focused — one feature, one fix
  - A PR touching 20+ files is hard to review well. Split it if you can.
- Self-review your own diff before submitting
  - You will catch ~30% of your own bugs just by reading it as a reviewer would
- PRs should have a short description: what changed, why, how to test it
- Assign a reviewer — don't let PRs sit unreviewed for days

What Happens Before a Human Reviews

~5 min

In professional teams, your PR is checked automatically before any human looks at it.
(This assumes you've already run your tests locally. CI is the safety net, not the first check.)



What those checks typically do:

✓ **Unit tests run —**	If any test fails, the PR is blocked from merging
✓ **Linter passes —**	Style violations surface automatically — not in human review comments
✓ **Build succeeds —**	The project actually compiles / starts up on a clean machine

On GitHub: this is GitHub Actions — a free feature configured by a YAML file in your repo. We'll set it up properly in the CI lecture. For now, just know it exists.

What to Look for in a Code Review

(In roughly this priority order):

- Correctness — does it do what it claims? Are edge cases handled?
- Clarity — can you understand what this code does in 30 seconds?
- Design — does it fit the architecture? Is anything doing too much?
- Test coverage — is new behavior tested? Are the tests meaningful?
- Error handling — what happens when things go wrong?
- Security — injection points, exposed secrets, missing authorization checks
- Performance — obvious problems ($O(n^2)$ loops, N+1 queries) — not micro-optimization
- Style — but only if your linter doesn't catch it automatically

How to Give Review Feedback

- Be specific — point to the exact line, not the general area
- Explain why it's a problem, not just that it is one
- Suggest a fix, or ask a question — don't just label
- Use "nit:" prefix for optional style preferences
  - nit: signals "this isn't blocking, just a thought"
- Review the code — never the person
  - "This function has three responsibilities" not "this is messy“

Examples:
❌"This is wrong."
✅"This will throw a NullPointerException if user is null — could we add a guard here?"
❌"I wouldn't do it this way."
✅"I've seen this pattern cause performance issues at scale — have you considered a cache? Happy to discuss."

Exercise: Code Review

```
def process_order(order_id, user):
    data = db.execute(
     f"SELECT * FROM orders WHERE id={order_id}"
    )
    if data != None:
        if user.role == 'admin' or user.id == data.user_id:
            data.status = 'processed'
            db.save(data)
            notify(user.email, data)
            log('Order: ' + str(order_id))
            return True
        else:
            return False
    else:
        return False
```

Your task:

- Find every issue in this code
- "Write" a comment for each one — as if leaving a real code review
- "Label" each: Bug · Security · Design · Style
- For each issue: state why it's a problem and suggest a fix
- Be ready to share 1–2 with the class

Hint: there are at least 6 distinct issues

Smells fixed: magic number, deep nesting, long method, unclear names, mixed concerns

Documentation as Communication

- Code says WHAT. Good documentation says WHY.
- Every public function deserves a docstring — not a novel, just intent + edge cases
  What does it do? What are the parameters? What can go wrong?
- Inline comments for non-obvious logic — not for obvious logic
  If you had to think for more than 10 seconds to write it, future readers will too
  ❌ # increment counter by 1 → tells you nothing the code doesn't already say
  ✅ # rate limiter resets on the minute boundary, not per-request → this is useful
- Your README is the front door of your project
  What is this? How do I run it? How do I contribute?
  An out-of-date README is a broken front door
- Don't leave dead comment blocks — delete old code, don't comment it out

Comments: Good vs. Noise

❌ **Noise — don't do this:**

```
# loop through users
for user in users:
    # check if active
    if user.is_active:
        # send email
        send_email(user)
```

✅ **Useful — explains the WHY:**

```
# Only notify users active in the last 30 days.
# Inactive users opted out of transactional email
# per the 2023 privacy policy update (see PR #241).
for user in recently_active_users:
    send_email(user)
```

Docstring example:

```
def calculate_refund(order_id, reason):
    """
    Calculates refund amount for a given order.
    Partial refunds apply if order has been partially fulfilled.
    Returns 0.0 if order is not refund-eligible (does not raise).
    Raises OrderNotFoundError if order_id does not exist.
    """
```

Writing Testable Code

Next lecture is Testing Fundamentals. This week: write code that can be tested.

- Pure functions are the easiest thing to test
  - Same input → always same output, no side effects, no global state
  - If a function reads from a file, a database, or a clock — it's impure
- Separate your logic from your I/O
  - The business logic (calculate, validate, transform) should be pure
  - The I/O (read from DB, write to file, call API) should be at the edges
- Dependency injection makes hard dependencies swappable
  - Pass dependencies in, don't create them inside the function
  - In tests, pass a fake/mock; in production, pass the real thing
- A function you can't test is almost always a function that's doing too much
  - If it's untestable — that's a design smell, not just a testing problem

Testable vs Hard to Test

❌ **Hard to test:**

```
def get_discount(order_id):
    order = db.get_order(order_id)
    if order.total > 1000:
        order.discount = 0.1
        db.save(order)
    return order.discount

# To test this you need a real (or mocked) DB,
# You can't test the logic in isolation
```

✅ **Testable**

```
def calculate_discount(order_total):
    if order.total > 1000:
        return 0.1
    return 0.0

# Test is trivial:
# assert calculate_discount(1500) == 0.1
# assert calculate_discount(500) == 0.0
# No DB, no mocks, no setup needed.
```

The logic is the same. The version on the right separates logic from I/O.
The I/O wrapper still exists; it just calls calculate_discount() instead of containing the logic.

Debugging as a Discipline

- Debugging is hypothesis testing — not random poking
  - State a hypothesis: "I think the problem is in the validation step"
  - Test it: add a print / set a breakpoint / isolate the case
  - Revise: if wrong, update the hypothesis. Don't keep the same print forever.
- Bisection: narrow the problem space in half each time
  - Binary search for bugs — eliminate half the code as the cause, then half again
- Rubber duck debugging — explain the problem out loud
  - To a person, to a duck, to a blank wall. The act of explaining often reveals the bug.
- Read the stack trace — don't skip it
  - The actual error is usually in the middle, not the first or last line
- Reproduce it reliably before trying to fix it
  - A bug you can't reproduce consistently is a bug you can't verify you've fixed

Example Stack Trace

```
Error: Cannot read properties of null (reading 'email')
    at sendOrderConfirmation(/app/services/notificationService.js:42:31)
    at processOrder (/app/services/orderService.js:78:5)
    at async OrderController.create (/app/controllers/orderController.js:23:7)
    at async /app/middleware/errorHandler.js:12:5


Node.js v18.17.0
```

A Debugging Workflow

1. Reproduce the bug reliably
   - Write the exact steps or a failing test case. If you can't reproduce it, stop here.
2. Understand what the code is supposed to do
   - Read the relevant functions carefully. Don't assume — verify.
3. Form a hypothesis about the cause
   - Be specific: "I think X is happening because Y"
4. Test the hypothesis — isolate, don't scatter
   - Add a targeted breakpoint or assertion. Not 20 print statements everywhere.
5. Fix, then verify
   - Does the original reproduction case now pass? Write a regression test.
6. If stuck for > 30 minutes — ask someone
   - Seriously. Explain the problem out loud. This is not admitting defeat.

What Incremental Progress Looks Like

Progress should be visible — not just to you, but in the repo

- Commits in the remote repo, PRs being opened and reviewed, issues being closed

The 4-milestone model for your implementation phase:

- 25% — scaffolding done: project structure, dependencies, CI config, hello-world build
- 50% — core flow works end-to-end, even if rough. You can demo the main path.
- 75% — all specified features implemented, edge cases handled, obvious bugs fixed
- 100% — tests passing, code reviewed, docs updated, demo-ready

The 50% milestone is the most important checkpoint

- If you can't demo your core flow at halfway, you're behind even if you have lots of code

Watch for common traps:

- Building in silos — nobody integrates until the last day
- Perfecting one part while ignoring others — scope management
- Not committing until it's "done" — commit working increments daily

Can you demo your core flow end-to-end right now, even roughly?

Before Next Class

Concrete actions for this week (for your project):

- Set up a linter and commit the config to your repo
- Add or update your CONTRIBUTING.md (even 4-5 lines is enough)
- Open at least one real PR — with a description — and have a teammate review it
- Add docstrings to every public function you write this week
- Find one function that mixes logic and I/O — refactor it so the logic is pure
- Write one test — any test — for any function in your codebase
  (We'll build on this heavily in Testing Fundamentals next week)
- If you hit a bug you can't solve in 30 minutes — ask a teammate (use the workflow)

Key Takeaways

- Design artifacts guide — they don't dictate. Update them when code diverges.
- Code quality is a team responsibility. Readable code is kind to your teammates.
- Automate style. Humans review logic.
- Learn to recognize smells. Trust the feeling that something is off.
- Comments explain WHY. Code says WHAT.
- Testable code = well-designed code. Separate logic from I/O.
- Debugging is systematic. Form a hypothesis. Test it. Revise.
- Progress is visible. Commit often. Integrate early.

Next Week:

Software Testing Fundamentals

Assignment: Weekly Standup Reflection

https://rrostock1.github.io/Ursinus-CS375/Assignments/StandupReflection

(Individual Assignment)
Due Feb 26 (and every week thereafter)

Assignment: Detailed Design Report

https://rrostock1.github.io/Ursinus-CS375/Project/Design

(Group Assignment)
Due March 26

Assignment: The Overdose

https://rrostock1.github.io/Ursinus-CS375/Assignments/Overdose

(Individual Assignment)
Due Apr 23